Schleifen – Fortgeschrittene Themen

Einführung in die Programmierung 1

TU Wien, Wintersemester 25

Was wurde zuletzt besprochen?

- Schleifen allgemein
- while-Schleife in Java
- do-while-Schleife in Java
- for-Schleife in Java

Was wird in diesem Foliensatz besprochen?

- Verschachtelte Schleifen
- Komplexere Kombinationen von Schleifen mit Verzweigungen

Was lernen Sie kennen?

- Verschachtelte Schleifen in Java
- Komplexere Schleifenmuster in Java
- Programmiermuster mit Schleifen

Verschachtelte Schleifen

Eine Schleife ist eine Anweisung

Daher kann eine Schleife auch in einer Schleife vorkommen usw.

Beispiel (2D-Muster ausgeben)

```java
int n = 3;
int m = 5;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        System.out.print("*");
    }
    System.out.println();
}
```

```
*****
*****
*****
```

Entwicklung der Zählvariablen

```
i:0 j:0
i:0 j:1
i:0 j:2
i:0 j:3
i:0 j:4

i:1 j:0
i:1 j:1
i:1 j:2
i:1 j:3
i:1 j:4

i:2 j:0
i:2 j:1
i:2 j:2
i:2 j:3
i:2 j:4
```

Weitere Beispiele für Ausgaben

```java
int n = 5;
for (int i = 0; i < n; i++) {
    for (int j = i; j < n; j++) {
        System.out.print("*");
    }
    System.out.println();
}
```

```
*****
****
***
**
*
```

```java
int n = 5;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        System.out.print("*");
    }
    for (int j = i; j < n; j++) {
        System.out.print("#");
    }
    System.out.println();
}
```

```
#####
*####
**###
***##
****#
```

```java
int n = 5;
for (int i = 0; i < n; i++) {
    for (int j = 0; j <= i; j++) {
        System.out.print("*");
    }
    System.out.println();
}
```

```
*
**
***
****
*****
```

```java
int n = 5;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        System.out.print(i + j);
    }
    System.out.println();
}
```

```
01234
12345
23456
34567
45678
```

Beispiel (Multiplikationstabelle, Angabe)

- Eine Multiplikationstabelle für das kleine Einmaleins in der folgenden Form ausgeben

*	1	2	…	9	10
1	1	2	…	9	10
2	2	4	…	18	20
3	3	6	…	27	30
4	4	8	…	36	40
5	5	10	…	45	50
6	6	12	…	54	60
7	7	14	…	63	70
8	8	16	…	72	80
9	9	18	…	81	90
10	10	20	…	90	100

Beispiel (Multiplikationstabelle, Lösung)

```java
public class Multiplication {

    public static void main(String[] args) {
        int n = 10;
        System.out.print("*\t");
        for (int i = 1; i <= n; i++) {
            System.out.print("\t" + i);
        }
        System.out.println("\n");
        for (int i = 1; i <= n; i++) {
            System.out.print(i + "\t");
            for (int j = 1; j <= n; j++) {
                System.out.print("\t" + i * j);
            }
            System.out.println();
        }
    }

}
```

Beispiel für komplexeres Muster – Angabe

- Folgendes Muster sollte erzeugt werden (für gegebenes n)

n = 2

```
v-v
-v-
```

n = 3

```
v---v
-v-v-
--v--
```

n = 4

```
v-----v
-v---v-
--v-v--
---v---
```

Beispiel für komplexeres Muster – Analyse

- Es werden n Zeilen ausgegeben
- Jede Zeile enthält 2*n – 1 Zeichen (Anzahl der Spalten bzw. Breite)
- Beispiel: Position der „v“-Zeichen für n= 4

```
v-----v
-v---v-
--v-v--
---v---
```

Zeile	Spalte links	Spalte rechts
1	1 (= Zeile)	7 (= Breite - 0 = Breite - Spalte links + 1)
2	2 (= Zeile)	6 (= Breite - 1 = Breite - Spalte links + 1)
3	3 (= Zeile)	5 (= Breite - 2 = Breite - Spalte links + 1)
4	4 (= Zeile)	4 (= Breite - 3 = Breite - Spalte links + 1)

Beispiel für komplexeres Muster – Lösung

```java
public class VPattern {
    public static void main(String[] args) {
        int n = 4;
        int width = 2 * n - 1;
        for (int row = 1; row <= n; row++) {
            for (int column = 1; column <= width; column++) {
                if (column == row || width - column + 1 == row) {
                    System.out.print('v');
                } else {
                    System.out.print('-');
                }
            }
            System.out.println();
        }
    }
}
```

Beispiel (Simulation)

Spiel beginnt mit einem Startbetrag (stake)

In jeder Runde wird 1 Euro eingesetzt

- 50 % Chance auf Gewinn von einem Euro (Einsatz zurück + 1 Euro)
- 50 % Chance auf Verlust des Einsatzes (- 1 Euro)

Es wird am Anfang ein Zielbetrag (goal) festgelegt

- Wird dieser erreicht, dann wird das Spiel beendet

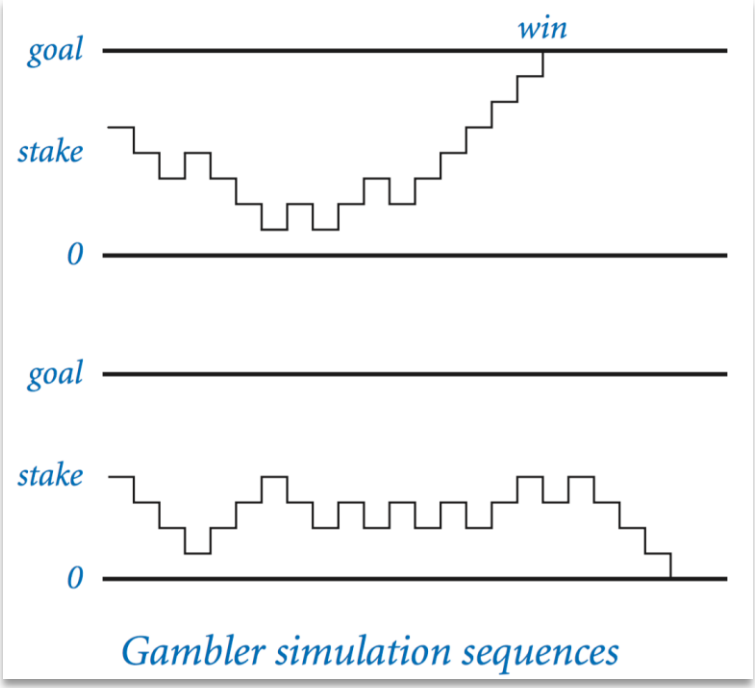
Beispiel (Simulation)

Simulation

- Zufälliger Entscheid, ob Gewinn oder Verlust
- Abbruch, wenn Zielbetrag erreicht oder kein Geld mehr vorhanden
- Mehrmals wiederholen (trials) und Mittelwerte berechnen

Code in TUWEL

- GamblingSimulation.java



https://introcs.cs.princeton.edu/java/13flow/images/gambler.png

Auswahl einer Schleife (1)

for-Schleife	while-Schleife
• Vorgegebener Start- und Endwert • Fixe Schrittweite	• Lediglich Endkriterium gegeben • Nicht vorhersagbar, wie oft die Schleifenanweisungen ausgeführt werden müssen, bis das Endkriterium zutrifft

Auswahl einer Schleife (2)

Transformation

Die verschiedenen Schleifenarten lassen sich auch ineinander transformieren

Anwendung

Abhängig vom Problem ist meist eine der Schleifen leichter ausformulierbar (lesbar)

Beispiele für Transformationen (beide Richtungen!)

while-Schleife	entsprechende do-while-Schleife

`while (ausdr)`
`  Schleifenanweisung;`
	`if (ausdr)`
	`  do`
	`  Schleifenanweisung;`
	`  while (ausdr);`

do-while-Schleife	entsprechende while-Schleife

`do`
`  Schleifenanweisung;`
`while (ausdr);`
	`Schleifenanweisung;`
	`while (ausdr)`
	`  Schleifenanweisung;`

for-Schleife	entsprechende while-Schleife

`for (ausdr1; ausdr2; ausdr3)`
`  Schleifenanweisung;`
	`ausdr1;`
	`while (ausdr2) {`
	`  Schleifenanweisung;`
	`  ausdr3;`
	`}`

Akkumulierende Schleifen

Schleifen, mit denen ein Ergebnis konstruiert wird

Schema

- Akkumulierende Variable vor der Schleife initialisieren
- Akkumulierende Variable in der Schleife verändern
- Nach der Schleife Ergebnis weiterverwenden

Akkumulierende Schleifen (Beispiele)

```java
int sum = 0;
int i = 1;
while (i <= 10) {
    sum += i;
    i++;
}
System.out.println(sum);
```

```java
int sum = 0;
for (int i = 1; i <= 10; i++) {
    sum += i;
}
System.out.println(sum);
```

Akkumulierende Schleifen (Beispiele)

```java
public class SpecialSum {

    public static void main(String[] args) {

        int sum = 0;
        int counter = 2;

        while (counter <= 100) {
            if ((counter % 2) == 0) {
                sum = sum + counter;
            }
            counter = counter + 1;
        }

        System.out.println("Sum: " + sum);
    }

}
```

Die Summe aller geraden Zahlen im Intervall [2, 100] berechnen und ausgeben

```java
public class Specialsum {

    public static void main(String[] args) {
        int sum = 0;
        for (int counter = 2; counter <= 100; counter += 2) {
            sum = sum + counter;
        }
        System.out.println("Sum: " + sum);
    }

}
```

Kürzere Variante mit for-Schleife – startet mit einer geraden Zahl und zählt immer 2 beim Update dazu

Zaunpfahlproblem – Motivation

- Beispiel
  - Die Zahlen von 1 bis n durch ein Komma getrennt ausgeben
- Falsche Lösung

```java
int n = 5
for (int i = 1; i <= n; i++) {
    System.out.print(", " + i);
}
System.out.println();
```

```
, 1, 2, 3, 4, 5
```

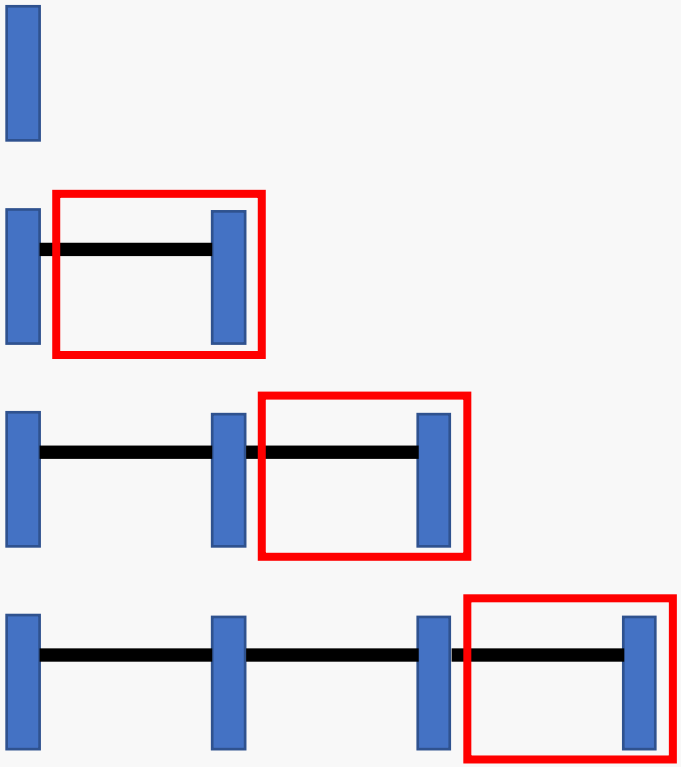
- Problem
  - n Zahlen werden ausgegeben, aber nur n-1 Kommas werden benötigt
  - `System.out.print(i + ", ");` löst nicht das Problem!

    Ergibt `1, 2, 3, 4, 5,`

Zaunpfahlproblem – Analogie

- Analogie Zaun
  - 4 Pfähle, 3 Streben
- Aufbau
  - Einen Pfahl fixieren
  - Wiederhole

    Strebe + nächsten Pfahl hinzugeben



Zaunpfahlproblem – Lösungen

- 1. Variante
  - Vor der Schleife Start fixieren

    Siehe vorherige Folie
- 2. Variante
  - Nach der Schleife abschließen
- 3. Variante
  - Mit Bedingung Ausgabe steuern

```java
int n = 5;
System.out.print(1);
for (int i = 2; i <= n; i++) {
    System.out.print(", " + i);
}
System.out.println();
```

```java
int n = 5;
for (int i = 1; i <= n - 1; i++) {
    System.out.print(i + ", ");
}
System.out.println(n);
```

```java
int n = 5;
for (int i = 1; i <= n; i++) {
    System.out.print(i);
    if (i < n) {
        System.out.print(", ");
    }
}
System.out.println();
```

```
1, 2, 3, 4, 5
```